OpenVLA Project/Research Explanation

This plain-language report, research narrative, methods log, results summary, limitations, and future work for technical details

Prepared for beginners to review and explanation

This report is written in simple English on purpose. The technical details are kept correct, but each term is explained in everyday language.

Executive summary

This project studies a robot control model called OpenVLA-7B. In simple words, OpenVLA-7B is a robot brain that looks at a picture and reads a sentence, then predicts how a robot arm should move. I tested whether small changes in wording can change the predicted movement. This matters because a real robot can become unsafe if it reacts too differently to small changes in language. [1][3]

The project produced two completed experiments. The first used a hand-built set of 40 BridgeData trajectories, called Manual40. The second used an automatically sampled set of 50 trajectories from a larger scripted BridgeData download, called Auto50. Together, these runs gave me two independent views of the same question: does changing the instruction change the action?

The answer was yes. Across both experiments, paraphrased prompts stayed closest to the original prompt, while contradictory and neutral prompts caused larger changes. Final-state images were usually more fragile than initial-state images. That means scenes near the end of a task were often easier to disturb with wording changes. This is a useful safety result even before adding a simulator or a physical robot loop.

1. Personal motivation and project story

I became interested in this topic because robot learning feels like the meeting point of software, language, and real-world action. A normal language model only writes text. A normal vision model only looks at images. A VLA model tries to do something much harder: look, understand, and act. That idea pulled me in because it feels like the beginning of robots that can follow natural instructions in the real world. [1]

I chose OpenVLA because it was openly available, strongly connected to current robot learning work, and practical enough to run locally with careful engineering. That made it a good research target for a student project: modern enough to matter, but open enough to test deeply. [3]

The project started from a simple question: if I tell the robot the same task in slightly different words, will it still choose almost the same action? At first this sounds small, but it is actually a safety question. A trustworthy robot should not act wildly differently just because the user said the same thing in a slightly different way. Recent papers now show that this is a real problem in VLA models, which means the question is timely and important. [4][5][6]

2. Research gap

The broad area is already active. The 2026 VLA survey shows that the field is moving quickly across architecture, datasets, training, and deployment. The LeRobot paper also shows how open tooling is helping robot learning become more reproducible. [1][2]

However, recent 2025-2026 papers show a clear weakness: many VLA models still struggle with language grounding. LangGap argues that some state-of-the-art models often ignore language more

than people expect. IGAR shows that contradiction failures can be reduced at inference time without retraining. BayesianVLA argues that dataset structure can create information collapse, where the model starts acting like a vision-only policy. SafeVLA, SAFE, and FailSafe show that safety, failure detection, and recovery are becoming central research problems. [4][5][6][7][8][9]

My project does not claim to be the first paper to say that language matters. That would be too strong. The real contribution is narrower and more honest: I performed a head-to-head comparison of paraphrased, contradictory, and neutral prompts on OpenVLA-7B, and I tested the effect separately on initial and final task states using both a manually curated set and an automatically sampled set. That combination gave a small but real niche contribution.

3. Why these matters in real life

If a home robot hears “put the cup in the sink” and “place the cup into the sink,” it should behave almost the same way. If it does not, that is a trust problem.

If a factory robot reacts differently to a contradictory or vague instruction, that can turn into lost time, broken parts, or unsafe motion.

If a healthcare robot assistant is given an unclear prompt, the safest action may be to pause, ask again, or move to a safe standby position. That means language robustness is not just an accuracy problem. It is also a safety and human-factors problem.

4. Technical setup in simple language

I ran the project inside WSL, which stands for Windows Subsystem for Linux. In simple words, WSL lets a Windows laptop run a Linux working environment. I used it because the OpenVLA stack and its Python packages behave more naturally in Linux than in normal Windows command line setups.

The main model was OpenVLA-7B in 4-bit mode on an RTX 2060 GPU. “7B” means the model has roughly seven billion parameters, which are the internal numbers the model uses to represent what it has learned. “4-bit” means the model weights were compressed to save memory, which made local inference possible on a smaller GPU. [3]

The first major failure was an attention-mask mismatch in the Hugging Face inference path. In simple words, the text tokens and the image tokens were not being counted together correctly during generation. That broke the model. I solved this by patching predict_action so it prepended a 256-token image mask before generation. This fix was the turning point that made local inference stable enough to continue.

Why does this matter? Because without that patch, the experiments would not have run at all. Research is not only about the final table. It is also about making the system work in a reproducible way.

5. Experimental design

I used four prompt conditions for each image: normal, paraphrased, contradictory, and neutral. I measured action drift in the first six action dimensions using L2 distance and cosine similarity. L2 distance is a simple measure of how far one action vector moves from another. Cosine similarity measures whether two vectors point in a similar direction even if their lengths differ. [1]

Manual40 experiment: I manually built 40 trajectory pairs from BridgeData-style tasks. Each trajectory had one initial frame and one final frame, so the total number of evaluated images was 80. This experiment gave stronger control over task wording and image selection.

Auto50 experiment: I downloaded and sampled 50 trajectories from a larger scripted BridgeData branch. Each trajectory again had one initial frame and one final frame, so the total number of evaluated images was 100. This experiment served as a broader validation set.

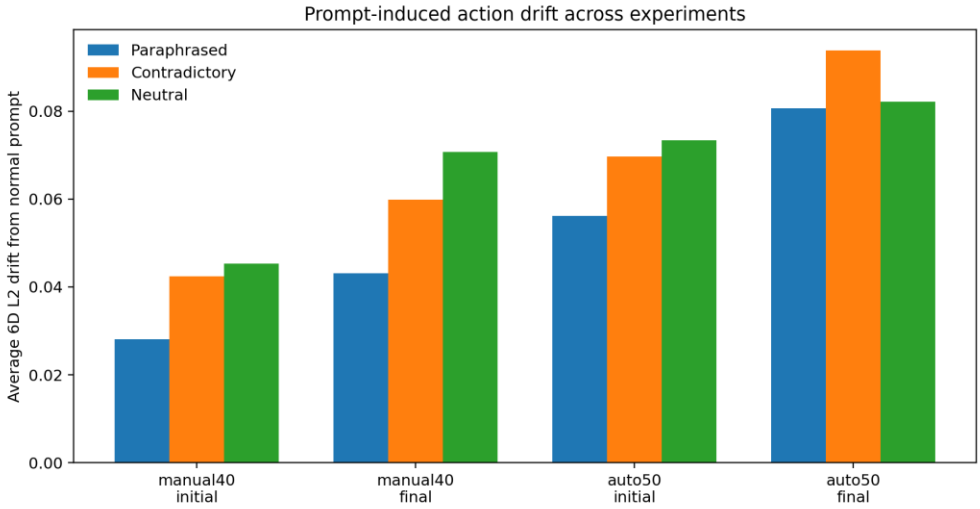

Figure 1. Prompt-induced action drift across the two completed experiments. Why does this matter? It shows that the same trend repeats in more than one evaluation setup.

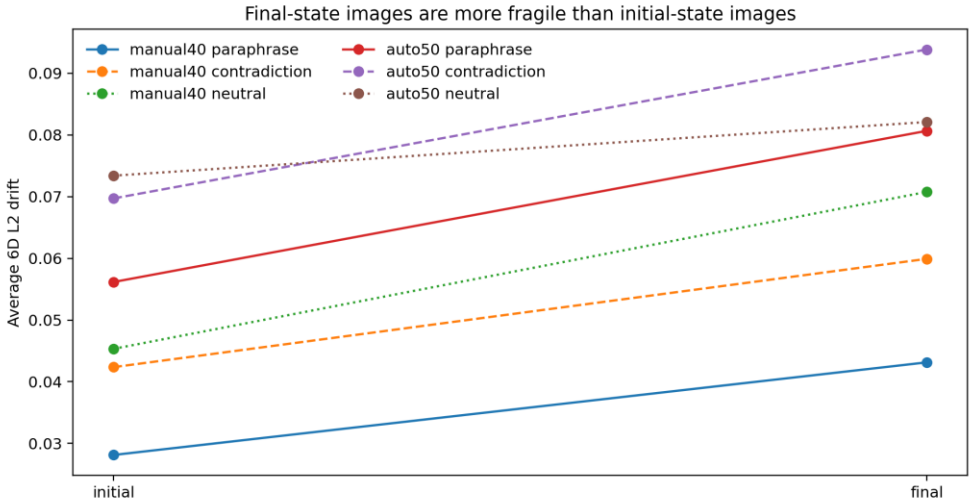

Figure 2. Final-state images are generally more fragile than initial-state images. Why does this matter? It suggests that scenes close to task completion may be especially sensitive to wording changes.

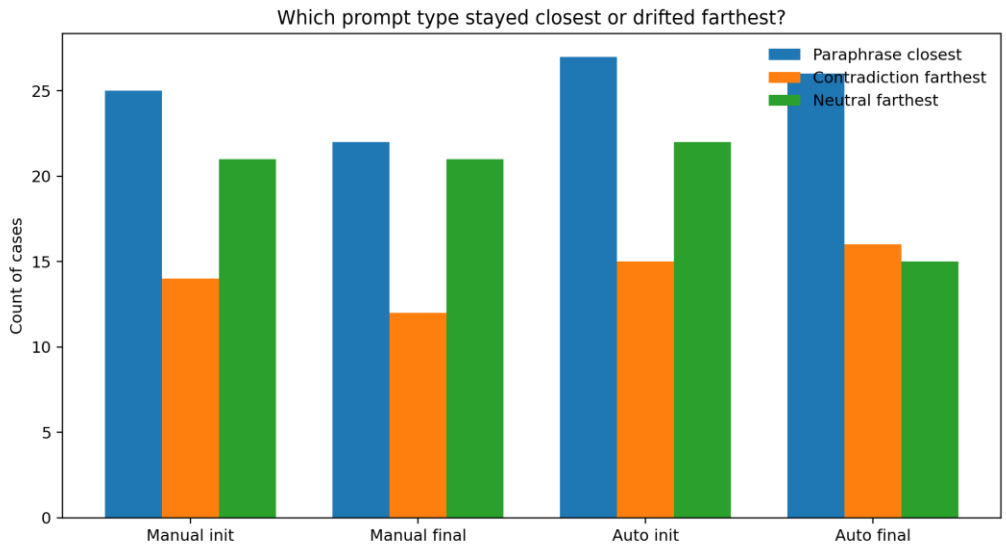

Figure 3. Counts showing which prompt type stayed closest or drifted farthest. Why does this matter? It gives an easy-to-read view of prompt behavior beyond averages.

6. Main quantitative results

Experiment	Group	n	L2 paraphrase	L2 contradiction	L2 neutral	Cos paraphrase	Cos contradiction	Cos neutral
manual40	initial	40	0.028126	0.042365	0.045330	0.678776	0.586046	0.425675
manual40	final	40	0.043141	0.059920	0.070772	0.480281	0.219040	-0.001856
auto50	initial	50	0.056164	0.069716	0.073405	0.483856	0.269209	0.170514
auto50	final	50	0.080668	0.093857	0.082103	0.386891	0.157202	0.220440

Table 1. Main summary metrics across both experiments.

Manual40 initial frames: paraphrases caused the least drift, contradictions caused more drift, and neutral prompts caused the most drift.

Manual40 final frames were more fragile than initial frames. Neutral prompts pushed the action farthest on average.

Auto50 initial frames repeated the same pattern: paraphrase stayed closest, while contradiction and neutral prompts moved farther away.

Auto50 final frames were again more fragile. Contradiction was the largest drift on average, while neutral was close behind.

7. Invariance and collapse counts

Experiment	Group	Exact all same	Near all same (<0.01)	Paraphrase closest	Contradiction farthest	Neutral farthest
manual40	initial	1	8	25	14	21
manual40	final	0	4	22	12	21
auto50	initial	0	1	27	15	22
auto50	final	0	1	26	16	15

Table 2. How often the model barely changed at all, and how often each prompt family stayed closest or moved farthest.

Why does this matter? These counts show that language sensitivity is not identical in every case. Sometimes the model nearly collapses and behaves similarly across prompts, but in most cases there is still meaningful drift.

8. Confidence intervals and reliability

Experiment	Group	Metric	Mean	95% CI lower	95% CI upper
manual40	initial	paraphrased	0.028126	0.016002	0.042951
manual40	initial	contradictory	0.042365	0.027787	0.059539
manual40	initial	neutral	0.045330	0.032649	0.059542
manual40	final	paraphrased	0.043141	0.030908	0.056137
manual40	final	contradictory	0.059920	0.042288	0.079732
manual40	final	neutral	0.070772	0.052691	0.090367
auto50	initial	paraphrased	0.056164	0.041880	0.071180
auto50	initial	contradictory	0.069716	0.053953	0.087871
auto50	initial	neutral	0.073405	0.060928	0.088257
auto50	final	paraphrased	0.080668	0.061054	0.103965
auto50	final	contradictory	0.093857	0.076429	0.112974
auto50	final	neutral	0.082103	0.068564	0.097324

Table 3. Bootstrap confidence intervals for the main drift metrics. Why does this matter? It shows that the ordering is not coming from only one or two lucky examples.

9. Where the project succeeded and where it failed

Succeeded: local OpenVLA inference was stabilized on a modest GPU through environment setup and the attention-mask patch.

Succeeded: the Manual40 branch produced a clean initial-versus-final comparison with strong and interpretable patterns.

Succeeded: the Auto50 branch validated the trend on a second dataset branch with 100 image rows.

Failed or remained incomplete: I did not yet close the loop with a simulator or a physical success metric. That means the current paper shows action drift clearly, but it does not yet prove task failure on a benchmark like LIBERO.

Failed or remained incomplete: I did not yet implement an active safety detector or recovery policy. That means I can diagnose the problem better than I can currently prevent it.

10. Limitations, future work, and what is still stopping the next step

No direct task-success study yet

Right now, the project measures changes in action vectors, not final task success. The main reason is time and infrastructure: building a clean simulator or replay evaluator that matches these image-only tests takes extra engineering. The obvious next step is to test the highest-drift cases in simulation and report success, failure, unsafe motion, and abstention rates.

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

No safety guard yet

The project does not yet stop the robot when prompt disagreement becomes too large. This is not solved yet because it needs threshold calibration and a clear fallback policy. The next step is to add a detector that compares normal versus alternate prompts and triggers re-prompting, safe retreat, or human clarification.

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

Single-model focus

The current study focuses on OpenVLA-7B. That is reasonable for a student project, but it limits the strength of the general claim. A stronger paper would repeat the same protocol on another VLA model and compare whether the drift pattern is model-specific or broader.

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

Image-only evaluation

The current pipeline uses still images and prompt variants. Real robots act over time. Extending the work to full trajectories, simulator rollouts, or real robot trials would strengthen the story.

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

Dataset scale

Manual40 and Auto50 are useful and real, but still small compared with the biggest benchmark papers. The study can be extended with larger automatic sampling, more task families, or benchmark suites that explicitly stress language grounding.

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

Why others have not solved this fully yet

This problem is hard because VLA datasets often contain visual shortcuts. In many datasets, the image already gives away the goal, so the model can partly ignore language and still do well. That makes language failures easy to hide. Recent papers like LangGap, IGAR, BayesianVLA, SafeVLA, SAFE, and FailSafe show that the community is only now starting to confront the problem directly. [4][5][6][7][8][9]

Why does this matter? Because this research paper should not only say what worked. It should also say what still needs to be built and why that part is genuinely difficult.

11. Why this can still become a conference paper

For this report, the work is already strong in my opinion. For a conference like CoRL, the diagnosis alone is probably not enough because recent papers already discuss language fragility and safety in VLA models. The strongest upgrade path is to add two things: (1) a small outcome study that links drift to success or failure, and (2) an inference-time safety mechanism that detects risky disagreement and triggers a fallback. That pivot would move the paper from “this problem exists” to “this problem can be detected and mitigated.” [4][5][6][7][8][9]

12. References

[1] Sapkota et al., 2026. Vision-Language-Action (VLA) Models: Concepts, Progress, Applications and Challenges. (https://arxiv.org/abs/2505.04769)

[2] Cadene et al., 2026. LeRobot: An Open-Source Library for End-to-End Robot Learning.(https://arxiv.org/abs/2602.22818)

[3] Kim et al., 2024. OpenVLA: An Open-Source Vision-Language-Action Model.(https://arxiv.org/abs/2406.09246)

[4] Hou and Zhao, 2026. LangGap: Diagnosing and Closing the Language Gap in Vision-Language-Action Models.(https://arxiv.org/abs/2603.00592)

[5] Zhang et al., 2026. Restoring Linguistic Grounding in VLA Models via Train-Free Inference Adjustment (IGAR). (https://arxiv.org/abs/2603.06001)

[6] Lian et al., 2026. BayesianVLA: Bayesian Decomposition of Vision Language Action Models via Latent Action Queries.(https://arxiv.org/abs/2601.15197)

[7] Zhang et al., 2025. SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning.(https://arxiv.org/abs/2503.03480)

[8] Gu et al., 2025. SAFE: Multitask Failure Detection for Vision-Language-Action Models. (https://arxiv.org/abs/2506.09937)

[9] Lin et al., 2025. FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models. (https://arxiv.org/abs/2510.01642)

[10] Brohan et al., 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control.(https://arxiv.org/abs/2307.15818)

[11] Open X-Embodiment Collaboration et al., 2025. Open X-Embodiment: Robotic Learning Datasets and RT-X Models.(https://arxiv.org/abs/2310.08864)